

Agents unlock new capabilities through Switching LoRA Adapters as a Tool (SLAaaT)

Kenneth Ge

Independent

San Francisco, CA, United States of America

kennethkouge@gmail.com

Abstract

Post-training can unlock new capabilities and improve performance on specialized tasks, but sometimes at the cost of catastrophic forgetting in other domains. This poses a problem in long agent trajectories that compose different capabilities. We reject this tradeoff by giving an agent a tool to switch between specialized LoRA adapters mid-trace. To test its effectiveness, we compose two synthetic coding tasks that are logically simple but require specialization. We find that this allows the model to solve problems it previously could not, that the model is able to switch autonomously (and find a new strategy that beats our human heuristic baseline on one task), and that this incurs an up to an 18x reduction in capability tax compared to an agent using only one specialized adapter. Our approach also substantially outperforms spawning subagents in both task capabilities (solving 4 of our hardest tasks versus none) and token usage (46.1x fewer tokens in some scenarios).

1 Introduction

For a given architecture and parameter budget, models often have to make tradeoffs between specialization and generality [Yue et al., 2025]. This is especially pronounced in small language models, which readily absorb new capabilities [Fu et al., 2023] but can forget old ones [Marek et al., 2026].

Fine-tuning lets small models match massive ones on specific tasks [Patil et al., 2023]. However, such models are currently used as subagent specialists with limited functionality [Belcak et al., 2025]. Even when exposed as tools in an agentic setting, these specialist models are invoked and switched to once [Shekar and Krishnan, 2025].

In this paper, we allow an agent to repeatedly hotswap its own LoRA adapters within the same trajectory (Figure 1). We find that the agent composes these swaps to solve new synthetic tasks without sacrificing overall capacity. With just one in-context example totaling 103 tokens, the model was able to learn when to use this tool, nearing the performance of our human heuristic switching on most tasks and exceeding on one. The model outperformed all other baselines, including staying on a single LoRA adapter, using a fused adapter fine-tuned on all tasks, Arrow [Ostapenko et al., 2024], and invoking subagents as a tool. Table 1 summarizes our results. All code, prompts, and data are available on Huggingface at <https://huggingface.co/kennethge123/autolora>.

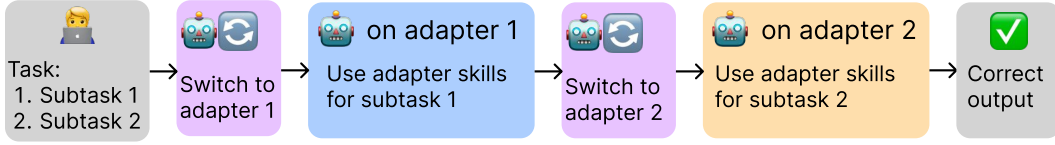


Figure 1: The agent switches its own LoRA adapter mid-trajectory, activating the right specialist for each subtask.

Table 1: Task passes (out of 50 per cell) across the four notation bindings. Shading scales with pass rate. Rows group into single-configuration arms, routed arms over the fixed {fauxjson, fauxthon} library, and the frontier reference.

Condition	json-python (control)	fauxjson-python	json-fauxthon	keystone (both)	Total /200
base	42	0	22	0	64
fauxjson (pinned)	0	0	0	0	0
fauxthon (pinned)	19	0	11	0	30
fused	0	0	0	0	0
arrow (token-level)	41	0	2	0	43
subagent	38	2	19	0	59
heuristic switch	45	18	30	8	101
autoswitch (model)	45	37	30	4	116
sonnet5 (reference)	49	8	16	2	75

2 Method

2.1 Task Design

A task that isolates whether post-trained skills compose should have the following properties:

- The necessary skills are absent from the base model and nontrivial to elicit via in-context examples (the capability is gained via post-training)
- The task decomposes into multiple stages that each require one skill, and success in a later stage is conditional upon success in an earlier one
- The subtasks are intrinsically easy, with equivalent counterparts that are already solvable by the model, so differences in performance are not about difficulty
- Task success is verifiable
- Training corpora for each subtask are fully disjoint

We designed a synthetic data processing pipeline with two steps: translating data from YAML to either JSON or a synthetic markup format called Fauxjson, and writing a program in either Python or Fauxthon to process this data and output the result as XML. Fauxjson and Fauxthon are just JSON and Python with their keywords/symbols cycled. For example, the symbols [{, }, :] might map to [}, :, {]. This shares the tokenizer, interferes with the model’s strong priors, and keeps the spec simple.

In total, our test suite included 5 different processing task types (lossless conversion, counting, aggregation, sorting, and grouping), with 10 different tasks per task type and 10 YAML inputs each (1 given to the model, 9 held out). We scored a 1 if the translated data file was correct and if the program produced a correct result on all 10 inputs, and 0 otherwise.

2.2 Adapter Design

Our fine-tuning data consisted of 20k synthetic YAML to Fauxjson translation pairs and 20k Fauxthon coding exercises transpiled from MBPP [Austin et al., 2021] (950 rows), Magicoder [Wei et al.,

2024] (9,050 rows), and synthetic programs (10k rows). All synthetic data was either transpiled programmatically or generated by Claude Sonnet 5; Fauxjson and Fauxthon were generated by our transpiler and verified to round-trip. Training details are in Appendix A.

The Fauxjson, Fauxthon, and fused adapters were each trained on 1 epoch. The fused adapter used a shuffled combination of both datasets.

2.3 Model and Sampling

We used Qwen3.6-35B-A3B, a model large enough to understand tool calls and solve non-trivial coding problems but still capacity constrained. For sampling, we gave the model 20 turns and a 4096 token budget, and we human-validated that these limits were only reached when models were stuck generating repetitive content. We kept thinking off.

2.4 Adapter Invocation and Tools

We used a standard subagent invocation tool where the host model writes a prompt. For our heuristic switching decisions, we monitored for `file_write` tools and switched to the adapter corresponding to the file extension. Autoswitch is a new tool where the model selects either base (no adapter), fauxthon, or fauxjson, and the harness switches the model accordingly.

3 Results

We answer three questions: do separately trained skills compose, can the model route them itself, and what does each configuration cost?

Table 1 summarizes end-to-end pass rates for every condition, and Figure 2 shows how much base capability each retains. Statistical notes in Appendix C.

Table 2: Testing our four single-configuration conditions on just fauxjson and fauxthon tasks outside of agentic tool calling confounds. Shading scales with score (higher is better).

Subtask	base	fauxjson	fauxthon	fused
fauxjson translation /50	0	23	0	44
fauxthon program tests /500	0	0	290	360

Composition We first verified that skills live in their respective adapters. We reran each of our four single-adapter configurations on just the Fauxjson translation tasks and the Fauxthon programming tasks. To remove the agentic harness confound, we had the model just emit a codeblock. We find that each adapter performs well at its respective task, and that the fused adapter actually performs by far the best on knowledge for both tasks (Table 2). However, fusion fails all agentic tasks (Table 1). Analyzing the traces, we find strong evidence that this is due to catastrophic forgetting: the fusion condition forgets Python and JSON, and meanwhile writes syntactically correct but semantically incorrect Fauxthon code.

Ideally, composition allows the probability of end-to-end success to be directly related to success on each subtask. For the human heuristic condition, we find that this largely holds true: the keystone score of $8/50$ nears the predicted $18/50 \times 30/50 \approx 11/50$, so the switching mechanism itself largely preserves continuity across the trace.

Routing Autoswitch beats our heuristic on Fauxjson-Python because it invented an innovative new policy: using the Fauxjson adapter to write Python code. This suggests models can already autonomously discover optimal routing choices. Meanwhile, Autoswitch’s actual score on the Fauxjson–Fauxthon (keystone) condition, $4/50$, is far lower than predicted by its performance on the individual Fauxjson–Python and JSON–Fauxthon tasks, $37/50 \times 30/50 \approx 22/50$, suggesting large capability gains remain on the table.

Costs Both the fused and fauxthon adapters incur a significant capability tax that switching largely avoids (Figure 2). Compared to the subagent condition, Autoswitch incurred approximately the same

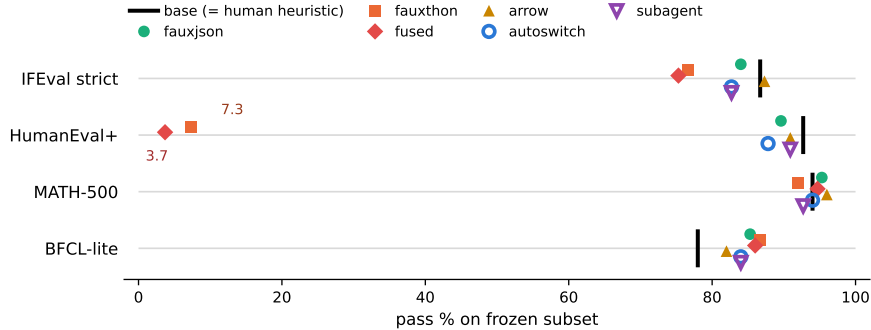


Figure 2: Retention of general capabilities. Pass rate on random frozen subsets of IFEval, HumanEval+, MATH-500, and BFCL-lite for each condition; the black bar marks the base model (equivalently, the human heuristic). The fused and fauxthron adapters pay a large capability tax, most severely on HumanEval+ (3.7% and 7.3%), whereas the routed conditions (autoswitch, subagent, arrow) stay close to base. This is the “capability tax” referred to throughout Section 3.

Table 3: Output tokens per task attempt. Shading scales with token count (more tokens is worse).

Condition	passed: mean	passed: median	failed: mean	failed: median
autoswitch (116 passes)	1,793	834	2,138	797
subagent (59 passes)	5,242	1,071	39,407	36,731

capability tax, both from routing unnecessarily. Autoswitch also used 1.3x fewer tokens for the median success compared to subagent, and 46.1x fewer tokens for the median failure (Table 3).

4 Limitations and Future Directions

Limitations Our tasks are synthetic by design: Fauxjson and Fauxthron isolate the composition question. All results use a single base model (Qwen3.6-35B-A3B), a single training seed, and one sample per task, so we report binomial confidence intervals (Appendix C). Our routing library contains two adapters and scaling to many adapters may elicit different behavior. Finally, the human heuristic is a fixed file-extension rule rather than an optimal policy, so “beating the heuristic” should be read as beating a strong hand-written rule, not an upper bound.

Future directions This paper establishes that models can effectively switch their own LoRA adapters under policies that sometimes beat human switching decisions. Future work could use reinforcement learning to train an even stronger routing policy. The functionality already exists to reuse the KV cache for different LoRA adapters, solving the biggest technical challenge [Li et al., 2025]. Future work could scale this method to production.

Acknowledgments

Thank you to Thinking Machines for supporting this research with a Tinker Research Grant.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.

- Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic AI. *arXiv preprint arXiv:2506.02153*, 2025. URL <https://arxiv.org/abs/2506.02153>.
- Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. Specializing smaller language models towards multi-step reasoning. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2301.12726>.
- Allison Li, Kristjan Greenewald, Thomas Parnell, and Navid Azizan. Efficient multi-adapter LLM serving via cross-model KV-cache reuse with activated LoRA. *arXiv preprint arXiv:2512.17910*, 2025. URL <https://arxiv.org/abs/2512.17910>.
- Martin Marek, Dongkyu Cho, Shikai Qiu, Rumi Chunara, Pavel Izmailov, and Andrew Gordon Wilson. Forgetting in language models: Capacity, optimization, and self-generated replay. *arXiv preprint arXiv:2605.26097*, 2026. URL <https://arxiv.org/abs/2605.26097>.
- Oleksiy Ostapenko, Zhan Su, Edoardo Maria Ponti, Laurent Charlin, Nicolas Le Roux, Matheus Pereira, Lucas Caccia, and Alessandro Sordoni. Towards modular LLMs by building and reusing a library of LoRAs. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2405.11157>.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023. URL <https://arxiv.org/abs/2305.15334>.
- Pavan C Shekar and Aswanth Krishnan. Adaptive minds: Empowering agents with LoRA-as-tools. *arXiv preprint arXiv:2510.15416*, 2025. URL <https://arxiv.org/abs/2510.15416>.
- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with OSS-Instruct. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2312.02120>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL <https://openreview.net/forum?id=40sgYD7em5>.

A Training and Evaluation Details

All adapters were trained via the Tinker API on Qwen3.6-35B-A3B for one epoch. Evaluation used

Table 4: LoRA training hyperparameters.

Base model	Qwen3.6-35B-A3B
LoRA rank	16
Learning rate	1×10^{-4} , 3% linear warmup, cosine decay to 0
Optimizer	Adam
Batch size	128 conversations
Epochs	1
Fauxjson training rows	20,000
Fauxthon training rows	20,000

20 turns and a 4,096-token budget per attempt with thinking disabled (Section 2.3). Test tasks are held out from all training data; each of the 50 tasks per condition uses 1 YAML input shown to the model and 9 held-out inputs for scoring.

B Compute

All training and evaluation ran on the Tinker API (Thinking Machines), except the Arrow baseline, which ran locally on a single NVIDIA DGX Spark. Producing the main results table (Table 1) took roughly two days of wall-clock time end to end.

C Statistical Notes

Every cell in Table 1 is a pass count out of $n = 50$ independent tasks from a single run (one seed=5, one sample per task). Treating each cell as a binomial proportion, the 95% Wilson interval half-width is at most ≈ 14 percentage points (at $p = 0.5$) and shrinks toward the extremes (e.g. 45/50: [79%, 96%]; 0/50: [0%, 7%]). The conclusions we draw—fusion and pinned adapters failing agentic tasks outright, and autoswitch’s total of 116 versus subagent’s 59—are separated by margins well outside these intervals. We did not run multiple seeds because of compute cost.

D Assets and Licenses

We use Qwen3.6-35B-A3B (Apache-2.0), MBPP [Austin et al., 2021] (CC-BY-4.0), and Magicoder [Wei et al., 2024] (MIT) as sources for transpiled training data. Our released assets—adapters, transpiler, task suite, prompts, and evaluation harness—are available at <https://huggingface.co/kennethge123/autolora> with a README describing setup and reproduction.

E Broader Impact

This work makes it cheaper to give a small model many narrow skills without paying a general capability tax, which lowers the barrier to deploying capable agents on modest hardware. That same property could make it easier to bolt undesirable specializations onto an open model, though it does not enable anything a full fine-tune could not already do; our released adapters target synthetic languages with no real-world use and pose no incremental risk.

F AI Usage

LLMs were used to help draft the appendix, format the $\text{\LaTeX}$ source, write captions, and make minor edits to the text. LLMs did not affect the methodology, results, or conclusions.